ROAM Presentation Script

Slide 1 — Title

Good morning everyone. I'm Ye Zihao. Today we present our final project **ROAM** — a retrieval-augmented optimization agentic model for spontaneous trip blueprint generation.

Slide 2 — Contents

We have five parts: problem, prior work, our method, evaluation and discussion.

Slide 3 — Background & Motivation

Why do we need an AI travel planner in the first place? Imagine that you are planning to travel, you first take three to five hours of research, cross-checking distances, reading reviews, and making trade-offs. So obviously people want to travel more easily and also more spontaneously, but let's face it, most of us are too busy to spend time planning. So the demand is clear: we need something easy to tweak on the fly.

Now you might ask — why not just ask ChatGPT? We tried that, and it fails because of hallucination. Take the issue above as an example, most models are language models, so when you ask it to visit somewhere like CityU, it does not know where it is, also it does not know like us that CityU closes at 6 pm on weekends, which shows the spatial bias and time-window ignorance. And third, their knowledge is stale, the model has no way to pull fresh information. These three problems are exactly what ROAM is designed to solve.

Slide 4 — Prior Works

There are three main approaches before we designed ROAM. ITINERA from KDD last year does good semantic retrieval, but only handles single-day trips with no hard time windows. Commercial platforms like Ctrip have huge POI databases, but they rely on manual editing — no natural-language interaction. And OR solvers give you optimal routes mathematically, but they don't understand a word you say.

So ROAM targets the white space: multi-day, hard time-window feasibility, category-aware dwell time — optimizer for math, LLM for language.

Slide 5 — Pipeline

Here comes the overall architecture of ROAM. We showed it in the proposal, but it does have some slight changes. Let me briefly walk you through it.

Stage one is semantic retrieval — we take the user's query and run embedding vector search against our POI database to find relevant attractions.
Stage two is spatial clustering — we group those candidates by geographic proximity so each day covers a coherent area, no cross-city zigzag.
Stage three is TSP plus time-window correction — we run simulated annealing to find the shortest route, then do greedy swaps to respect opening hours.
Stage four injects RAG context — weather from wttr.in, road distances from OSRM, crowd patterns — so the plan reflects real-world conditions.
Stage five is LLM narrative generation — daily themes, route descriptions, and reasoning.

The key difference from what we showed in the proposal is that these functions are now exposed to the LLM as tool-calling methods, enabling ReAct-style replanning.

Slide 6 — Replanning Agent

Why do we need a separate replanning agent? Because the initial plan is static, but real trips are not. Users want to say things like ”swap the park on Day 1 for a museum” — in plain English, on the fly.

The problem is, if you manually edit the plan, you lose ROAM's optimizer guarantees — the TSP order breaks, time windows might conflict. So our solution is a ReAct-style function-calling agent. The LLM autonomously picks which tools to call — search, replace, reroute, time-feasibility check — and every edit runs through the exact same optimizer pipeline used at planning time. The result is natural-language revision with planner-grade feasibility.

Slide 7–9 — Examples

Here is an example of our interface. Users can obtain a complete itinerary plan for the trip. This page shows the route, transportation advice, and timeline suggestions.

This page shows detailed description of each recommended POIs for enhancing spots understandings.

This page shows the route map for users to reference. All these modules can be edited by the model based on the user's requirements, to deliver a more satisfying plan.

Slide 10 — Experimental Setup

For evaluation, we designed nine natural-language queries across four cities — Fuzhou, Guangzhou, Shenzhen, and Shanghai — all three-day plans. The queries cover six scenarios: distance constraints, time minimization, time-window scheduling, contextual awareness, and spatial coherence. The results are grouped into four categories shown in the following pages.

We compare three methods. Bare LLM has no retrieval, no optimization, no RAG — pure language model output. Baseline agent has search but no spatial optimization — basically ITINERA-style retrieval plus our prompt. And ROAM is our full model.

Slide 11 — Six Metrics

We initially set up about fifteen metrics to evaluate the model's performance, but we narrowed it down to six shown on the slide.
As shown, POI counts total stops — more is richer.
Semantic match (SM) measures alignment with user intent.
Transit ratio (TT) is commute time over available time — lower means tighter connections.
City exploration (CE) is convex hull coverage — broader spatial spread.
Day balance (DB) is the variation across days — lower means even load.
Route compactness (RC) is walk-equivalent distance per segment — lower means less travel cost.

Slide 12 — Results (Sunny & Rainy)

This slide shows the comparison across sunny and rainy-day scenarios. Two things stand out. First, POI count — ROAM dominates, averaging 28 stops in sunny scenarios versus 17 for Bare and 16 for Baseline. Second, rainy-day adaptation — ROAM shifts toward indoor POIs while keeping exploration high, proving the RAG module actually works.

Slide 13 — Results (cont.)

On the continuation of the table, two more highlights. City exploration — ROAM covers substantially more urban area while staying coherent; Baseline collapses to about one percent. Route compactness — ROAM is around 400 to 700 meters per segment; Baseline is 1300 to 1900 — that's the TSP optimization at work.

Bottom line: ROAM wins or ties on all six metrics, with large margins on POI, CE, and RC.

Slide 14 — Failure Cases & Limitations

ROAM fails in three main scenarios. Ambiguous requests that need clarification — our agent currently can't ask back. No matching POI for the requested category — if all museums are closed, we have no graceful fallback. And conflicting constraints like eight stops in a day with impossible time windows.

For future work: our evaluation is offline only, we haven't measured live latency. Real-time traffic and queue times aren't integrated yet. There's no personalization loop from user feedback. And token cost scales with plan size — each replan burns several LLM turns.